

PBL 03: 서버리스와 보안

정하은

1. 개요

클라우드 컴퓨팅 기술의 발전은 웹 개발의 패러다임을 지속적으로 변화시키고 있다. 특히, 서버리스(Serverless) 아키텍처는 개발자가 인프라 관리에 대한 부담을 덜고 핵심 비즈니스 로직에 집중할 수 있게 하면서 빠르게 확산되고 있다.

본 보고서는 이러한 서버리스 아키텍처의 개념을 이해하고, 이 새로운 환경이 가져오는 보안 취약점과 이에 대한 대응 방안을 제시하는 것을 목표로 한다. 기존 웹 개발 프레임워크와 서버리스를 비교하고, OWASP Serverless Top 10을 기반으로 위협을 분석하며, 실질적인 클라우드 보안 모범 사례를 포함하여 내용을 구성했다.

2. 웹 개발 프레임워크와 서버리스의 차이점

2.1 웹 개발 프레임워크

- **정의** : 웹 애플리케이션을 효율적으로 개발하기 위한 구조적 기반과 기능을 제공하는 소프트웨어 환경이다. 건축에 비유한다면 미리 설계된 건축 구조물로, 개발자는 이 구조물 위에 원하는 기능을 추가하고, 인터리어(UI/UX)를 꾸미는 데 집중할 수 있도록 하는 것이다.
- **특징**
 - MVC(Model, View, Controller), MVVM 등 **아키텍처 패턴**을 제공하여 코드의 유지보수성을 높여준다.
 - 라우팅, 데이터베이스 연동, 세션 관리, 인증 등의 **핵심 기능이 내장되어** 있어 개발자가 번거로운 기초 작업 없이 바로 핵심 로직에 개발 착수할 수 있다.
 - 잘 정의된 **컴포넌트나 모듈을 재사용**하여 반복 작업을 줄이고, 코드 재사용성 및 개발 속도를 향상할 수 있다
- **예시** : Django(Python), Spring(Java), Express.js(Node.js) 등

2.2 서버리스

- **정의** : 서버 인프라의 프로비저닝, 관리, 확장을 **클라우드 제공자가 담당**하고, 개발자는 **이벤트 기반으로 실행되는 함수를 배포**하는 아키텍처이다. 하지만 인프라 운영은 추상화되지만, 함수 배포·권한·이벤트 연결·관측성 설정 같은 애플리케이션 운영 책임은 여전히 남아있다. 건축에 비유하면 더 이상 건물을 짓거나 관리를 할 필요가 없이 필요한 기능이나 모듈만 만들어서 관리자에게 넘기면 알아서 만들어주는 구조이다.
- **특징**
 - 요청이 있을 때만 함수가 실행되어(**이벤트 기반**) 유휴 상태의 서버 비용을 없애준다.
 - 사용자가 함수를 호출하고 실행한 시간과 같은 **사용량을 기반으로 과금**이 된다. (종량제 모델)
 - 요청이 급증하면 클라우드 제공자가 자동으로 함수 인스턴스를 확장하고, 요청이 줄어들면 다시 축소한다. 개발자가 직접 서버의 성능을 모니터링하고 관리할 필요가 없어진다.
 - 서버리스 함수는 메모리, CPU, 최대 실행 시간 등 일부 **런타임·리소스 제약**이 존재한다.
- **예시** : AWS Lambda, Google Cloud Functions, Azure Functions

2.3 웹 개발 프레임워크와 서버리스 비교표

구분	웹 개발 프레임워크	서버리스
인프라 관리	개발자가 직접 서버 배포·관리	클라우드 제공자가 전담
실행 방식	상시 실행 서버 기반	요청 시 함수 실행
확장성	수동 또는 자동 확장 설정 필요	자동 확장 기본 제공
비용 모델	상시 서버 비용 발생	실행 시간·호출 수 기반 과금
개발 집중도	서버·애플리케이션 모두 고려	애플리케이션 로직 중심
보안 책임 범위	인프라+애플리케이션 모두	애플리케이션·권한 설정 중심

3. 서버리스의 취약점과 대응방안

3.1. OWASP Serverless Top 10

취약점	서버리스 특이점	영향	대응방안
Injection	API 외에도 DB 변경, 스토리지 업로드, 이메일, IoT 이벤트 등 다중 입력 경로	데이터 삭제·변조, 계정 탈취	입력 화이트리스트, ORM, 최소권한, 런타임 방어
Broken Authentication	인증 없는 내부 함수 트리거 가능	민감 데이터 유출, 로직 위변조	클라우드 제공 인증 서비스 활용(AWS Cognito, Firebase Auth)
Sensitive Data Exposure	환경변수·/tmp 잔여파일 노출	API 키·계정정보 유출	Secret Manager·KMS 사용, HTTPS
XXE	XML 파싱 시 외부 엔티티 허용	코드·환경정보 유출	안전한 파서, 엔티티 해제
Broken Access Control	과도한 IAM 권한 부여	자원 삭제·변경, 서비스 중단	함수별 최소권한 설정, 자동 점검

Security Misconfiguration	공개 스토리지, 노출된 키, 긴 타임아웃	정보 유출·DoS·비용 손실	보안 스캔, 타임아웃 최소화
XSS	이메일·로그 등 다양한 저장소 경유	브라우저 데이터 탈취	출력 인코딩, CSP
Insecure Deserialization	JSON, Pickle, node-serialize 등	코드 실행, 자원 탈취	타입 검증, 취약 라이브러리 점검
Known Vulnerabilities	취약 3rd-party 라이브러리 포함	SSRF 등	의존성 스캔·업데이트
Insufficient Logging	제한된 클라우드 로그	침해 탐지 지연	모니터링+추가 로깅 시스템

3.2. 추가 취약점과 대응방안

- DoS : 동시 실행 한도/디스크(/tmp)/백엔드 용량 소진 → 예약/계정 동시성 한도, 백엔드 쿼터 관리
- DoW(지갑 고갈) : 과금구조 악용 → 예산 알림, 사용 한도, 스토틀링
- Insecure Shared Space : 재사용 컨테이너의 /tmp 잔여물 → 사용 후 삭제, 임시키 짧은 TTL
- Business Logic/Flow : 이벤트 순서·아이덴티티 검증 부재 → 이벤트 서명, Idempotency 키, 상태 확인 로직

4. 안전한 서버리스 플랫폼 도입을 위한 구체적인 고려사항

앞서 살펴본 OWASP Serverless Top 10의 취약점들은 서버리스 환경의 특성 때문에 발생하는 고유한 위험들이었다. 이러한 위험을 효과적으로 방어하기 위해 클라우드 제공업체인 AWS와 Google Cloud는 다양한 보안 모범 사례와 전용 서비스를 제공하고 있다. 이 장에서는 앞서 정의한 취약점들을 염두에 두고, 각 클라우드 플랫폼에서 제공해 주는 보안 고려사항들을 구체적으로 다루어 보고자 한다.

4.1 AWS Lambda 보안

- 네트워크 보안
 - 함수는 가능하면 **VPC 내부**에서 실행하여 외부 접근을 최소화하도록 한다.
LAMBDA_INSIDE_VPC 규칙은 이 설정을 강제하여, 함수가 퍼블릭 인터넷이 아닌 내부 네트워크를 통해서만 다른 리소스와 통신하도록 만든다.
 - 민감한 데이터베이스나 내부 API에 접근하는 함수는 반드시 **프라이빗 서브넷**에 배치하여

보안을 강화해야 한다.

- **구성 및 권한**
 - 각 함수에 필요한 **최소 IAM 권한**만 부여하도록 한다. IAM 정책에 ***:***을 금지하고 각 함수가 필요한 **s3:GetObject** 나 **dynamodb:PutItem** 같은 특정 권한만 갖도록 IAM 정책을 세밀하게 작성해야 한다.
- **안정성 및 가용성**
 - **DLQ(Dead Letter Queue)**는 함수 실행에 실패한 이벤트를 **SQS(Simple Queue Service)**나 **SNS(Simple Notification Service)**에 보관하는 기능이다. 이는 실패 원인을 추적하고, 데이터 유실을 방지하는 안전장치 역할을 한다. 이러한 DLQ 설정으로 실패 이벤트를 안전하게 보관하도록 한다.
 - **LAMBDA_VPC_MULTI_AZ_CHECK** 와 같은 규칙을 통해 VPC 연결 함수가 서로 다른 AZ의 2개 이상 서브넷을 사용하는지 점검하여 함수가 **멀티 AZ(Availability Zone)**에 걸쳐 배포되도록 하면, 장애 발생 시에도 서비스 지속되도록 할 수 있다.
- **런타임 관리**
 - **LAMBDA_FUNCTION_SETTINGS_CHECK**와 같은 규칙은 허용되지 않은 런타임 사용을 자동으로 감지한다.
 - 사용 종료(EOL)된 런타임은 즉시 최신 버전으로 업그레이드하도록 한다.

4.2 Google Cloud Functions / Cloud Run 비밀 관리

- **Secret Manager 사용**
 - API 키, 비밀번호 등 민감정보의 평문 **환경 변수/코드 하드코딩**은 금지하고, **Secret Manager**를 통해 환경 변수(고정 버전) 또는 볼륨 마운트(최신 버전/회전)로 제공한다.
 - **환경 변수 전달** : 배포 시에 고정 버전을 참조하므로, 버전 고정 필요 시 유용하게 사용 가능하다.
 - **볼륨 마운트** : 항상 최신 버전을 참조할 수 있어 비밀 순환(Secret Rotation) 정책을 적용하기에 유리하다.
- **권한 관리**
 - Secret Manager에 접근할 수 있는 **roles/secretmanager.secretAccessor** 역할만 함수에 할당하여, **최소 권한**만 부여되도록 한다.
 - 기본 서비스 계정은 광범위한 권한을 가질 수 있으므로 대신 **사용자 관리형 서비스 계정**을 사용하는 것을 권장한다.
- **운영 절차**
 - 비밀 접근 경로·권한에 대한 주기적 점검이 필요하다.
 - 민감정보는 코드·Git 저장소에 절대 포함되지 않도록 한다.
 - 접근 허용된 비밀 목록을 정기적으로 검토하고 필요 시 삭제하도록 한다.

4.3 공통 보안 원칙

1. 아키텍처 설계

- 서버리스 환경에서는 모든 함수 호출이 이벤트 소스 및 트리거 인증·검증 절차를 거쳐야 한다.
- 각 함수의 권한과 데이터 접근 범위를 명확히 구분하도록 한다.

2. 데이터 보안

- 클라우드 제공자가 제공하는 KMS(Key Management Service), Key Vault, Secret Manager 등과 같은 클라우드 제공 암호화 서비스를 적극 활용하여 데이터 암호화 키를 안전하게 관리할 수 있다.
- HTTPS/TLS를 통해 암호화된 전송을 한다.

3. 코드 보안

- 모든 입력 데이터를 검증해야 한다. (화이트리스트, 스키마 검증 등 활용)
- 출력 시 HTML·JavaScript 인코딩을 적용한다.
- 취약 라이브러리를 스캔하고 주기적으로 업데이트를 해야 한다.

4. 운영 및 모니터링

- AWS CloudTrail, Azure Monitor, GCP Cloud Logging 등 기본 모니터링 도구와 별도의 SIEM 연동하여 로그를 통합 관리하여 함수 호출 패턴이나 비정상적인 활동을 효과적으로 탐지하도록 한다.

5. 비용 및 가용성 관리

- 서버리스의 종량제 과금 방식은 DoW(Denial of Wallet, 지갑 고갈) 공격에 취약하여 이를 방지하기 위해 호출 제한(Throttling)을 하고, 예산 알림 설정을 활용하여 비정상적인 비용 증가를 조기에 감지해야 한다.
- 고가용성 확보를 위한 멀티 리전 배포를 하도록 한다.

5. 결론

서버리스를 통해 개발자에게 인프라 관리라는 무거운 짐을 덜어주며 생산성을 극대화할 수 있었지만, 보안의 책임이 사라지는 것은 아니었다. 오히려 책임의 형태가 인프라 전체를 지키는 것에서 함수 하나하나의 보안을 철저히 하는 것으로 변화했다. 과거에는 방화벽과 네트워크 경계를 지키는 것이 중요했다면, 이제는 함수 코드 자체의 보안 취약점과 함수 간의 권한 설정을 세밀하게 관리하는 것이 중요해진 것이다.

특히, 서버리스의 종량제 과금 모델은 DoW(Denial of Wallet, 지갑 고갈) 공격이라는 독특한 보안 위협을 만들어냈다는 것이 인상 깊었다. 이는 공격자가 무분별한 함수 호출로 서비스 예산을 소진시켜 비즈니스를 마비시킬 수 있음을 의미한다. 따라서 함수 호출 제한(Throttling) 및 예산 알림 설정은 선택이 아닌 필수적인 보안 전략이 되어야 할 것이다.

결론적으로 서버리스 환경의 보안은 개발 단계부터 시작되어야 한다. 즉, 서버리스를 안전하게 도입하기 위해서는 DevSecOps 문화 정착과 자동화된 보안 점검 도구 도입이 필수이며, 모든 트리거·이벤트 소스를 잠재적 공격 경로로 보고 방어 전략을 세워야 한다. 클라우드 제공자가 제공하는 강력한 보안 도구들을 적극적으로 활용하고, 최소 권한 원칙을 철저히 지키며, 지속적인 모니터링을 통해 함수 하나하나를 안전하게 관리하는 것이 서버리스 시대에서 개발자에게 필요한 보안 역량이라고 생각한다.

참고 자료

1. OWASP. "OWASP Serverless Top 10". OWASP Foundation,
<https://owasp.org/www-project-serverless-top-10/>.
2. AWS. "AWS Lambda 보안 모범 사례". AWS Documentation,
https://docs.aws.amazon.com/ko_kr/config/latest/developerguide/security-best-practices-for-lambda.html.
3. Google Cloud. "Cloud Functions / Cloud Run 비밀 관리". Google Cloud Documentation,
<https://cloud.google.com/functions/docs/configuring/secrets?hl=ko>.